TempoSync | Proof of Concept

Technical feasibility and hackathon demo · Rev. 2 (October 15, 2026)

Executive concept

TempoSync is an activity-aware adaptive fitness DJ. Rather than playing a static playlist, it chooses the next permitted track, or transitions early, in response to a user's workout goal, current exercise state, preferences, and pace signals. Decisions come from a deterministic, explainable rule engine, not a trained model.

POC hypothesis: a simple decision engine can make intelligible, timely track-selection decisions from workout events, and a browser audio player can execute smooth or deliberate transitions, including landing a song on its drop, with licensed/local demo audio.

Problem and differentiator

During runs or strength sessions, energy needs change within a workout. A static playlist may not align with a runner drifting from a target pace or a lifter transitioning from exertion to rest.

Distinctive value: combine activity-specific taste (e.g., hip-hop for lifting, lo-fi for running), stated goal, workout state, and adaptive switching that enters a new song at its drop when the user needs a push. Do not claim that adaptive workout music has never existed or that a faster song necessarily improves performance.

POC scope

- Web demo with activity choice: running or strength training.
- Setup form for goal, favorite genres/artists, and seed tracks (text only); seed artists are matched against the catalog where possible. Chat-style prompts are an optional stretch.
- Curated local audio catalog with 12–16 authorized demo tracks (minimum 8), tagged by BPM, energy (1–5), genre, duration, and, for high-energy tracks, a hand-tagged drop timestamp (dropAtSec).
- Browser player with play/pause, current track, next recommendation, and transitions (immediate cut or configurable 2–4 second crossfade), plus starting a track at a chosen offset for drop-sync.
- Running simulation: target pace, current pace, pace drift (stretch: live phone GPS pace feeding the same events); strength simulation: start set, complete set, rest, next exercise.
- Decision explanation: show why a track was selected and whether a switch was triggered.
- No real smartwatch data, commercial music streaming, audio modification (seeking to a timestamp is allowed), or trained ML required.

Two demo stories

Running pace drift

Runner chooses lo-fi/jazz and target 10:00 min/mile. Simulation changes current pace from 10:00 to 12:00. Once the drift is at least 0:30 min/mile and has lasted 15 seconds, the engine selects a higher-energy eligible track and crossfades in so its drop lands as the fade completes. If exertion is flagged high, favor steady/recovery music instead of escalation. Running faster than target holds the current energy.

Strength exercise transition

User selects hip-hop, three sets of triceps extensions then bench press. 'Set complete' starts rest mode and fades into a calmer song from its beginning; 'Start next exercise' cues a higher-energy track. A manual 'Need a boost' control triggers a quick transition that lands on the next track's drop, without implying rep detection.

Architecture and data contracts

React + TypeScript + Vite frontend; Web Audio API with two HTMLAudioElements or audio nodes for crossfade; Python FastAPI backend for optional decision API. For the smallest POC, recommendation rules can run locally in TypeScript and be extracted behind a service interface.

Shared contracts (identical in the MVP document): Track {id,title,artist,genre,bpm,energy,durationSec,dropAtSec?,audioUrl,licenseNote}. Profile {id,preferencesByActivity,seedArtists,seedTracks,intensity}. WorkoutPlan {activity,goal,targetPace?,exercises?,restSec?}. WorkoutEvent {type,at,payload}. Decision {action:'keep'|'switch',trackId?,transition:'cut'|'crossfade'|'none',startAtSec?,reason,confidence?}. Store state locally; avoid a database initially.

Recommendation pipeline: filter licensed catalog by activity taste and genre; exclude currently playing track when switching; score candidates by energy target, BPM proximity, genre/seed-artist preference, and repetition penalty; apply cooldown (30 seconds) and hysteresis (pace drift of at least 0:30 min/mile sustained 15 seconds); for escalations and boosts, start at dropAtSec minus the crossfade length. Parameters are demo defaults, not validated training advice.

Decision rules and safety

Running: compare current pace (simulated or GPS) to goal; slower-than-goal sustained drift may raise desired energy by one level; on-target holds track; faster-than-goal holds energy and never escalates; recovery/high exertion overrides and reduces desired energy. Pace is min/mile (larger number means slower).

Strength: active set favors user-selected workout energy; set-complete enters rest and requests lower energy with crossfade; start-set raises energy. Never infer safe lifting limits or encourage pushing through pain, dizziness, or excessive exertion.

Switch only if eligible track exists; otherwise keep current track and explain. Respect pause state, cooldown, and user override. BPM is a music descriptor, not a direct measure of running cadence or performance.

Acceptance tests

- Running drift of at least 0:30 min/mile sustained 15 seconds triggers exactly one explainable recommendation, not repeated rapid skips.
- Brief pace noise does not trigger a switch.
- Running faster than target does not trigger escalation.
- Complete-set event triggers recovery track and audible crossfade.
- New exercise/start-set returns to workout energy.
- A boost or escalation to a drop-tagged track starts at dropAtSec minus the crossfade length; untagged tracks start at 0.
- User genre preferences and matching seed artists constrain choices where possible; a missing genre yields an explicit fallback.

- Audio plays only after user gesture (browser autoplay policy), and transitions work with bundled demo tracks.
- Safety/recovery flag blocks aggressive escalation.

Demo and success measures

Five-minute live script: choose running + lo-fi; play music; simulate pace drift; show the transition, drop landing, and reason; reset; choose lifting + hip-hop; complete set; demonstrate crossfade into rest; press 'Need a boost' to show a drop-synced switch. If GPS is built, show a short pre-recorded outdoor clip rather than relying on indoor signal.

Instrument event-to-decision time, decision-to-audio-start time, skipped transitions, and user override count. Do not assert exercise outcomes or motivational improvements without user testing.

Risks and boundaries

Spotify Premium does not by itself guarantee access to raw audio, arbitrary mid-track switching, or crossfading via third-party APIs. Validate platform terms and playback SDK limitations before committing; use properly licensed local demo tracks for the POC.

No copyrighted track alterations, no microphone-based breathing detection, no wearable integrations, no claims of clinical-grade monitoring. Personal preference learning is a future hypothesis, not a trained model in this prototype.

GPS pace is noisy indoors and at low speeds; smooth it, keep the simulator as the demo default, and fall back automatically. HackOHI/O allows pre-built components but asks teams to be transparent with judges about what was built during the event; record pre-event work in the README.

Build plan and handoff

Timeline: HackOHI/O 12 runs October 24–25, 2026 (24 hours). Suggested split — pre-event: audio catalog with drop tags, player, event simulator, decision engine and unit tests. Event hours 0–8: workout screens, crossfade + drop-sync. Hours 8–14: GPS pace (stretch) and accessibility. Hours 14–20: polish, deploy, README. Hours 20–24: demo rehearsal, Devpost submission, buffer. Shift items to match what is actually finished before the event.

Solo: implement player → event controls → recommendation rules → drop-sync → onboarding → tests → GPS stretch. Two people: Developer A owns React player, UX, crossfade and drop-sync; Developer B owns event schema, recommendation logic, GPS pace, FastAPI and tests. Agree on JSON interfaces before parallel work.

AI coding tool instruction: implement exactly this scope using the shared contracts above, use local authorized audio assets or clearly labeled placeholders, include README, sample fixtures, unit tests, and a one-command development start. Do not invent dropAtSec values; flag untagged tracks. Do not silently substitute Spotify streaming.